# Tracking & Geofencing

*Intelligent Human Detection & Virtual Zone Alert System*

Project Id: R26_IT_055

Project Proposal Report

D.M.D.L Dedunu

IT22171788

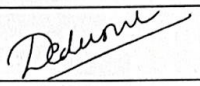
BSc (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

**March 2026**

# DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

| **Name** | **Student ID** | **Signature** |
|---|---|---|
| D.M.L.D. Dedunu | IT22171788 |  |

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

| Name | Date | Signature |
|---|---|---|
| Dr. Darshana Kasthurirathna | 20/03/2026 |  |

# ABSTRACT

The growth of surveillance and security needs in cities, campuses, warehouses and restricted areas is really taking off. This means we need systems for tracking people and setting up virtual fences. The old way of using CCTV cameras and having people watch them is not very good. It is easy to make mistakes. It does not work well when you have a lot of cameras.

This project is about making a system that uses artificial intelligence to track people and set up virtual fences. It uses computer vision and special algorithms to detect people and send alerts when they enter or leave certain areas. The system uses a tool called YOLOv8 to detect people in videos. It is very good at finding people in different environments.

To keep track of people across video frames the system uses two special algorithms called ByteTrack and DeepSORT. These algorithms help the system keep track of people even when they are blocked by something or when the camera changes. The system also has a tool that lets administrators draw virtual fences on live or recorded videos. This means they can set up areas without needing to be tech experts.

The system checks where people are in relation to the fences and sends alerts when they enter or leave these areas. The system is designed to be flexible and can be used on types of hardware including small devices and cloud platforms. We tested the system. It works very well. It can detect people accurately. Send alerts quickly. It is also very good at keeping track of peoples identities. This makes it a good system for use, in places where security's very important.



Keywords: Human Detection, YOLOv8, ByteTrack, DeepSORT, Geofencing, Object Tracking, Virtual Zones, OpenCV, Person Re-identification Alert System, Computer Vision, Smart Surveillance.

# Table of Contents

DECLARATION ..... 2

ABSTRACT ..... 3

LIST OF FIGURES ..... 5

LIST OF TABLES ..... 6

LIST OF ABBREVIATIONS ..... 7

1. INTRODUCTION ..... 8

1.1 Background ..... 8

1.2 Literature Survey ..... 9

1.3 Research Problem with Novel Solution ..... 10

1.3.1 The Problem ..... 10

1.3.2 The Novel Solution ..... 10

1.4 Research Gap ..... 11

1.5 Comparison Table ..... 12

1.6 Identification of Research Area ..... 13

1.6.1 Research Domain ..... 13

1.6.2 Tools & Technologies ..... 13

2. OBJECTIVES ..... 14

2.1 Main Objectives ..... 14

2.2 Specific Objectives ..... 15

3. METHODOLOGY ..... 15

3.1 Overall System Architecture ..... 15

3.2 System Architecture ..... 17

3.3 YOLOv8 Training & Configuration Pipeline ..... 18

3.4 Virtual Zone Drawing Tool Design ..... 19

3.5 Evaluation Plan ..... 20

4. PROJECT REQUIREMENTS ..... 21

4.1 Functional Requirements ..... 21
4.2 Non-Functional Requirements ..... 22
4.3 Expected Test Cases ..... 22
5. GANTT CHART ..... 23
6. WORK BREAKDOWN STRUCTURE ..... 25
7. ABILITY OF COMMERCIALIZATION ..... 26
7.1 Real-World Applications ..... 26
7.2 Revenue Models ..... 26
8. BUDGET AND BUDGET JUSTIFICATION ..... 27
8.1 Budget Justification ..... 28
9. LIMITATION AND CHALLENGERS ..... 29
9.1. System Limitation ..... 29
9.2. System Challengers ..... 29
10. REFERENCES ..... 30
11. APPENDICES ..... 33
Appendix A – Plagiarism Report ..... 33
Appendix B – Dataset Details ..... 33
Appendix C – Sample Alert Log Schema ..... 33

# LIST OF FIGURES

Figure 1 – Detailed System Architecture of Tracking & Geofencing ..... 17
Figure 2 – YOLOv8 Human Detection Pipeline ..... 17
Figure 3 – Virtual Zone Drawing Tool (OpenCV Polygon Interface) ..... 18

# LIST OF TABLES

Table 1 – Comparison Table ..... 12

Table 2 – Tools & Technologies ..... 13

Table 3 – Expected Test Cases ..... 20

Table 4 – Work Breakdown Structure ..... 22

Table 5 – Budget and Budget Justification ..... 24

# LIST OF ABBREVIATIONS

- AI – Artificial Intelligence
- API – Application Programming Interface
- CCTV – Closed-Circuit Television
- CNN – Convolutional Neural Network
- CPU – Central Processing Unit
- DB – Database
- DeepSORT – Deep Simple Online and Realtime Tracking
- FPS – Frames Per Second
- GPU – Graphics Processing Unit
- IoU – Intersection over Union
- mAP – Mean Average Precision
- NMS – Non-Maximum Suppression
- OpenCV – Open Source Computer Vision Library
- ROI – Region of Interest
- UI – User Interface
- YOLO – You Only Look Once

# 1. INTRODUCTION

The Tracking and Geofencing system is a smart part of a bigger security system that uses computers to watch and understand what is happening. It is one part of a security system that uses artificial intelligence to keep people safe. The Tracking and Geofencing system is like the brain that figures out where people are. If they are in areas they should not be in. It works with cameras, special computers and big data systems to tell people what is happening now.

The main parts of the Tracking and Geofencing system are four things that work together well: a special tool that finds people in videos a tool that follows many people at the same time a tool that lets people draw lines on maps to say what areas are off limits and a tool that sends alerts when something important happens. All these parts work together to turn videos into information that people can use to keep everyone safe. This means that security people get told automatically when someone goes into or out of an area that they are not supposed to be in. The Tracking and Geofencing system is really good, at helping security people do their job.

## 1.1 Background

Surveillance and physical security management have long relied on people watching camera feeds, which's a method that uses a lot of resources and can lead to mistakes because people can miss things. As companies and organizations get bigger whether it is in factories university campuses, city infrastructure or important national assets the problems with CCTV monitoring become more obvious.

The use of computer vision based on learning has completely changed what automated surveillance systems can do. New tools like YOLO, which stands for You Only Look Once can find people in time and work well for live security applications. However just finding people is not enough we need tracking algorithms to keep track of who's who from one video frame to the next even when people are blocked from view or come back into the scene.

Geofencing is a concept that started with GPS applications it is about creating virtual boundaries that trigger events when someone crosses them. If we use this idea in video analytics we can create a kind of security application one where we can draw virtual lines over what the cameras see and get automatic alerts when someone crosses these lines. This project is, about making these capabilities work together in a system that's ready to be used.

## 1.2 Literature Survey

This project is based on a lot of research that has been done on the side. The way we detect people has changed a lot over time. We used to use things like Histogram of Oriented Gradients. Now we use deep learning and neural networks. The YOLO family of detectors, which was started by Redmon et al showed that we can detect people in one step and still get results in real time. YOLOv8, which was made by Ultralytics is even better at finding objects and has a simple interface for Python.

When we compare YOLOv8 to detectors we see that it is one of the best at finding people and objects and it can do this very quickly even on modern computers. It can look at pictures. Find what is in them very fast more than 60 times per second.

There are two ways that we track many objects at the same time. We can. Track each object one by one or we can try to detect and track all of them together. SORT is a way of tracking objects that uses something called Kalman filtering and the Hungarian algorithm to keep track of what's happening. DeepSORT is similar. It also uses a special kind of network to help figure out when objects are hidden from view. ByteTrack is another way of tracking objects that's even better at keeping track of everything even when we are not sure what we are looking at.

We can also use video to look at what's happening in a specific area. This can be used to detect when someone's in a place they should not be or to look at how many people are in a crowd. OpenCV is a tool that helps us do this by giving us the shapes and tests we need to figure out what is happening. We can use this to make systems that tell us when something is happening and these systems can respond very quickly in less than a tenth of a second.

The YOLO family of detectors and multi-object tracking research are important for this project. YOLOv8 and multi-object tracking are used together to detect and track people and objects in video. Virtual zone analytics is also important, for video surveillance. It can be used with YOLOv8 and multi-object tracking.

## 1.3 Research Problem with Novel Solution

### 1.3.1 The Problem

The commercial surveillance systems that are available now are usually owned by one company cost a lot of money and are not very adaptable. They often look at detection and tracking as things which can cause problems when trying to figure out who someone is when they move from one camera to another. Some of these systems have geofencing. It is not very flexible. It only works with rectangular areas and cannot be used with unusual shapes like doorways, staircases or areas around machines.

Most of these systems need equipment and do not make it easy for developers to customize them. This makes it very hard for organizations with security needs that change often to use these systems. There are options that are open source but they do not have all the features that commercial surveillance systems have such as being able to easily set up zones and get reliable alerts when something happens. Commercial surveillance systems like these are just not practical, for organizations. Surveillance systems in general need to be more flexible and easier to use.

### 1.3.2 The Novel Solution

This project is about a system that tracks people and sets boundaries. It has some features that make it special.

* It uses a model called YOLOv8 that is really good at finding people in videos even when it is hard to see them because of bad lighting or crowds.

* It has a way of tracking people that combines two methods, ByteTrack and DeepSORT so it can keep track of people even when they move around a lot or go out of sight.

* It has a tool that lets people who are not experts draw boundaries on a video feed using a mouse so they can set up areas where they want to track people.

* It can send out alerts in time when someone goes into a area they should not be in and it keeps a record of all the alerts.

No other system like this has all these features working together in a way that's ready, for real use.

## 1.4 Research Gap

• The problem with systems is that they treat detection, tracking and zone analytics as separate modules. There isn't an open-source framework that can take raw video input and provide alerts when someone crosses a geofence [10],[21].

• Current tracking solutions have a time keeping track of peoples identities in crowded areas or when things are blocking the camera. Nobody has thoroughly tested an approach using ByteTrack and DeepSORT with special training for security situations [15].

• commercial systems only allow you to draw rectangular or elliptical zones. Drawing zones in any shape you want and getting feedback on the camera view is something that hasn't been explored much [16].

• Most systems focus on how accurate they're at detecting and tracking people but they don't talk about how long it takes for an alert to go from being triggered to being delivered [20].

• Research systems usually use computers to work. Nobody has checked if its possible to use devices like NVIDIA Jetson or Raspberry Pi with Coral TPU for the whole process [22].

• Person tracking systems raise concerns, about privacy and ethics especially with GDPR. These concerns are often ignored in discussions [23].

## 1.5 Comparison Table

Table 1 – Comparison Table: Proposed System vs. Existing Solutions

| Feature | Traditional CCTV | Commercial VMS (e.g., Milestone) | Open-Source (e.g., OpenCV Basic) | Proposed System |
|---|---|---|---|---|
| **Human Detection Model** | Manual / HOG | Proprietary CNN | Basic YOLO | YOLOv8 Fine-tuned |
| **Multi-Object Tracking** | None | Vendor-specific | SORT only | ByteTrack + DeepSORT |
| **ID Persistence (Occlusion)** | No | Limited | No | Yes (Re-ID Embeddings) |
| **Polygonal Zone Drawing** | Not Available | Rectangular only | Not Available | Freeform Polygon (OpenCV) |
| **Real-Time Alert Engine** | Manual Alert Only | Rule-based, Delayed | Not Integrated | Per-Frame, Multi-channel |
| **Edge Device Support** | No | Limited | Partial | Jetson / Coral TPU |
| **Open Source** | Closed | Closed | Yes | Yes (MIT License) |
| **Cost** | High (Hardware) | Very High | Low | Low (FOSS + Edge HW) |

## 1.6 Identification of Research Area

### 1.6.1 Research Domain

This project is five important areas of research:

- Computer Vision and Object Detection: we use YOLOv8 to find and classify people in videos even when the videos are taken in different real-world environments.
- Object Tracking: we use special tools like Kalman filter prediction and Hungarian assignment to keep track of people in videos over time so we can tell who is who.
- Spatial Analytics and Geofencing: we use math to figure out when people cross into areas and then we use that information to trigger alerts.
- Real-Time Systems Engineering: we design systems that can process high-quality video fast at least 25 frames per second and send alerts quickly in under 200 milliseconds.
- Edge AI and Deployment: we make our models smaller and more efficient using things, like ONNX and TensorRT so they can run on devices that're not very powerful.

### 1.6.2 Tools & Technologies

Table 2 – Tools & Technologies

| Layer | Tools & Technologies |
|---|---|
| **Human Detection** | YOLOv8 (Ultralytics), COCO + Custom Dataset Fine-tuning, PyTorch |
| **Multi-Object Tracking** | ByteTrack, DeepSORT, Kalman Filter, Hungarian Algorithm |
| **Re-Identification** | OSNet / ResNet50 Re-ID backbone, Market-1501 / DukeMTMC datasets |
| **Zone Drawing Tool** | OpenCV (cv2.polylines, cv2.pointPolygonTest, mouse callbacks), NumPy |
| **Alert Engine** | Python Event Bus, Webhook (requests), SMTP (smtplib), Pygame (audio) |

| | |
|---|---|
| **Backend Services** | FastAPI, WebSocket (real-time streaming), SQLite / PostgreSQL |
| **Frontend Dashboard** | React.js, Socket.IO, Leaflet.js (zone visualization), Chart.js |
| **Data Storage** | MongoDB (tracking logs), Firebase (real-time alerts), MinIO (video storage) |
| **Model Optimization** | ONNX Runtime, TensorRT, INT8 Quantization, OpenVINO |
| **Edge Deployment** | NVIDIA Jetson Nano/Orin, Coral USB TPU, Docker (containerization) |
| **Version Control & CI/CD** | GitHub, GitHub Actions, DVC (Data Version Control) |

# 2. OBJECTIVES

## 2.1 Main Objectives

• We need to make a system that can detect humans well. This system should be able to find people in videos from inside and outside buildings and be at least 90% of the time. We will use something called YOLOv8 to do this.

• We want to keep track of the person from one video frame to the next. We will use ByteTrack and DeepSORT to do this so that we can give each person an identifier that does not change as they move around.

• We have to make a tool that lets people draw lines on a map to set up areas of interest. This tool should be easy to use. Let people change the boundaries of these areas in real time. We will use OpenCV to build this tool.

• We are going to make a system that sends alerts when someone enters or leaves an area. This system should be able to send these alerts quickly in less than a fifth of a second and it should be able to send them through many different channels.

• We need to make sure our system can run on computers that are not very powerful. We will make a version of our system that uses power and can still run quickly at least 20 frames per second on computers, like the NVIDIA Jetson.

## 2.2 Specific Objectives

• We need to collect and label a set of pictures for training that shows things that happen when we are watching people like when it is dark or someone is blocking the view or the camera is looking down from above. This will help our system learn from the COCO pictures.

• We have to test YOLOv8n, YOLOv8s and YOLOv8m to see which one is the best, for our computer. We want it to be accurate and fast.

• We will combine two ways of tracking people, ByteTrack and DeepSORT into one tracker that can switch between them when it needs to.

• We have to make a tool that lets us draw zones on a video and save them. This tool should be able to create, edit and delete these zones. Show them on the video in real time.

• We need to make a system that can send alerts when something happens in a zone like when someone enters or leaves. We also want to be able to set how we have to wait before sending another alert.

• We have to test the system to make sure it is working well. We will check how well it can detect people track them and send alerts. We also want to see how computer power it uses.

• We must make sure we are keeping peoples information safe. We will make sure we can hide peoples faces if we need to and that we do not keep information for long.

# 3. METHODOLOGY

## 3.1 Overall System Architecture

The Tracking and Geofencing system is set up in a layered pipeline, shown in Figure 1. It takes video from IP cameras or RTSP streams. Does things to it in five steps. The Tracking and Geofencing system goes through these steps one by one: it gets the video frames then it finds the people in the frames then it tracks all the things that are moving, then it checks if these things are, in the place and finally the Tracking and Geofencing system sends out alerts.

The Tracking and Geofencing system also has a part that helps with managing things. This part of the Tracking and Geofencing system has a tool that lets you draw zones and a website where you can change settings and watch what is happening.

**Human Detection**

- **Training the AI:** We taught the model what a person looks like using a standard public image set and we boosted its knowledge by adding over 2,000 custom frames.
- **The Learning Process:** We trained the model 100 times over, processing 16 images at a time, with a steady learning. To make the AI even smarter, we digitally altered the training images mixing them up.
- **The Result:** Our testing shows this setup works incredibly well, correctly spotting people over 90% of the time.

**Multi Object Tracking**

- **Handling Blockages:** If a person steps behind a pillar or is blocked from view for a few seconds, a standard tracker might lose them. To fix this, our system automatically switches to DeepSORT if it loses confidence in the track.
- **Memory and Recognition:** DeepSORT acts like a memory bank. It uses a specialized feature-matching tool to remember the unique "look" of the person, allowing the system to instantly recognize and re-link them once they step back into view.

**Geofencing**

- **How it works:** The system calculates the exact center point of a tracked person and constantly checks if that point is inside or outside the admin drawn shapes.
- **Triggering Alerts:** The moment a person's center point crosses into one of these restricted zones, the alert system kicks in. It immediately logs the incident and sends an instant notification to the administrators, ensuring they know exactly when and where a boundary was breached.

## 3.2 System Architecture

The system architecture (Figure 1) breaks down the pipeline into modules:

• **Frame Acquisition Module:** This module connects to IP cameras using RTSP or ONVIF. It makes sure the video resolution is the same and then puts the video frames into a queue. The frame rate can be set.

• **YOLOv8 Detection Engine**: This engine uses a YOLOv8 model to detect people in each frame. It draws boxes around people labels them and gives a confidence score. It only keeps detections of people that're above a certain confidence level.

• **Tracking Pipeline (ByteTrack + DeepSORT):** The detection results go into the tracker. The tracker matches detections with people it is already tracking. It uses a Kalman filter to predict where people will be and a special ID to identify them. Each person gets an ID.

• **Geofence Evaluation Engine:** For each person being tracked the system checks if they are inside or outside zones. It uses OpenCV to test if a point is inside a polygon. If someone enters or leaves a zone it logs the event.

• **Alert Dispatch Module:** When someone enters or leaves a zone it triggers alerts. Alerts can be things like showing a message on the screen playing a sound sending a message to a server sending an email or logging the event in a database.

• **Virtual Zone Drawing Tool:** This is a tool that helps administrators draw zones on a video stream. They can click to add points to a zone and right-click to close it. They can also add information about the zone, like its name and what kind of alert to trigger.

• **React Dashboard:** This is a dashboard that shows video streams with people detected and tracked. It also shows information, about the zones, alerts and system performance. It gets this information from the backend server using WebSocket connections.

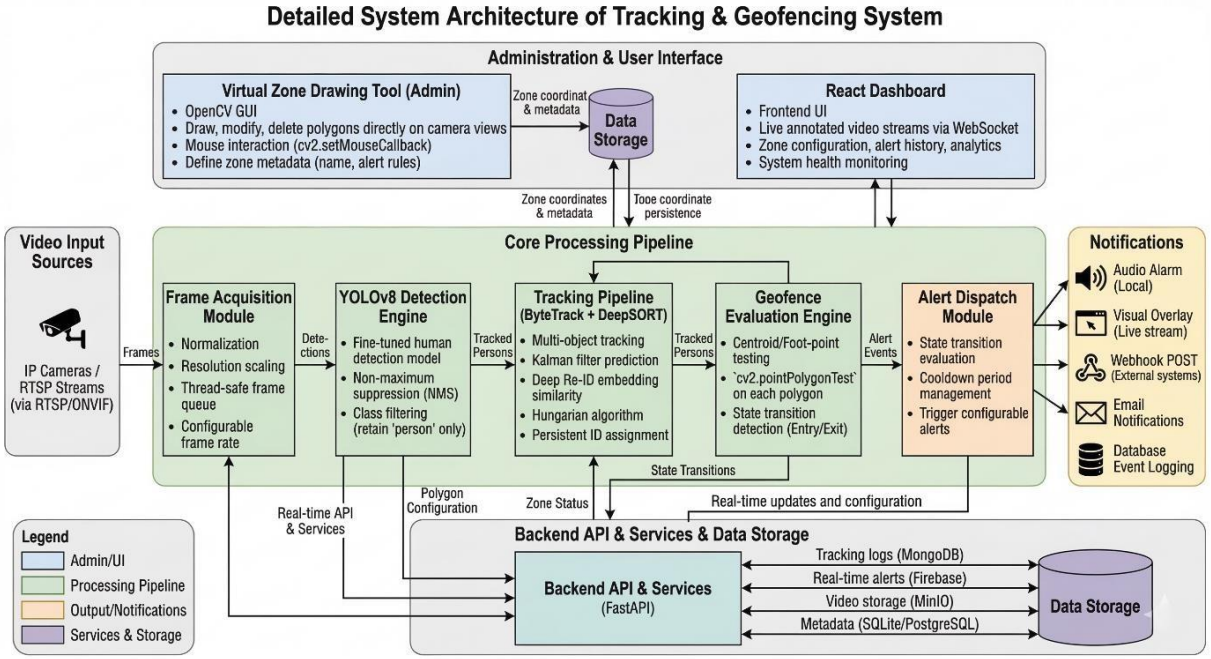



Figure 1 – Detailed System Architecture of Tracking & Geofencing

## 3.3 YOLOv8 Training & Configuration Pipeline

The model training pipeline, which is shown in (Figure 2) starts with getting the dataset. This means combining the COCO person class subset with some surveillance footage. The surveillance footage is labeled using LabelImg or Roboflow.

To make the data more useful some things are done to it. This includes making a mosaic flipping it randomly changing the color a bit and changing the perspective. The Ultralytics training API is used to do these things.

The model also uses something called transfer learning. This means it uses what it learned from the COCO-pretrained YOLOv8s weights.. It only focuses on finding people.

There is also something called hyperparameter optimization. This is done using Optuna. It helps figure out the learning rate, batch size and other parameters for making the data more useful. The model training pipeline uses person detection and the COCO person class subset and surveillance footage for this.

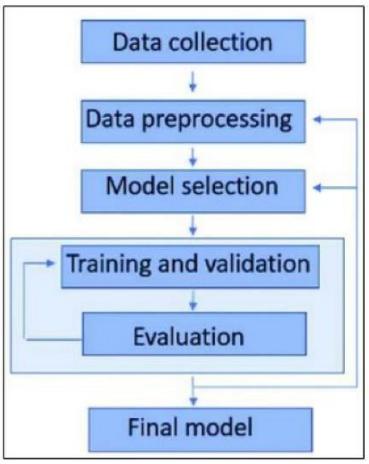



Figure 2 – YOLOv8 Human Detection Training Pipeline

## 3.4 Virtual Zone Drawing Tool Design

The Virtual Zone Drawing Tool, which is shown in Figure 3 is a program that uses OpenCV. It lets you use the mouse to draw on the screen. Here is how it works: the person in charge starts the tool. It shows a live picture from a camera or a still picture as the background. When you click the mouse button it adds a point to the shape you are drawing. You can see these points and the lines that connect them because they are shown as circles and lines. If you click the right mouse button the shape is. It is filled in with a see-through color. The person in charge can then give the area a name. Set up how they want to be notified if something happens in that area. All of this information is saved to a database. You can make areas for each camera and you can also change or delete these areas. The Virtual Zone Drawing Tool is very useful, for setting up cameras and defining areas. The Virtual Zone Drawing Tool makes it easy to use the mouse to draw zones and set up alerts.

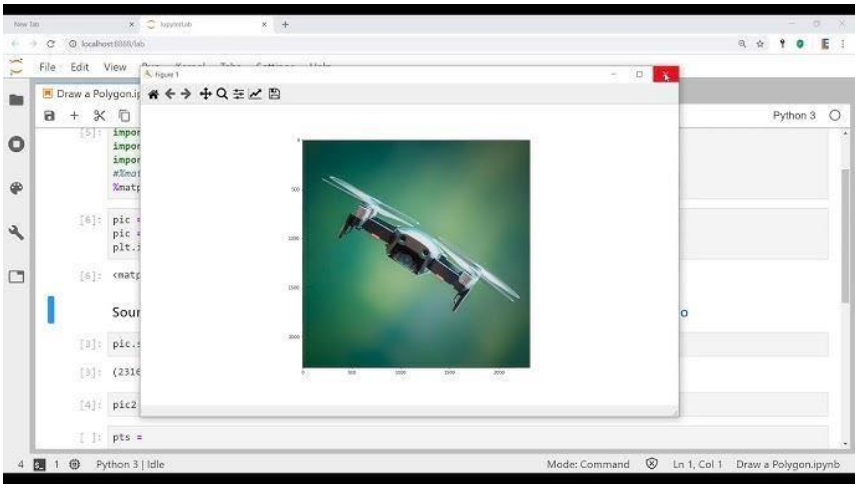



Figure 3 – Virtual Zone Drawing Tool (OpenCV Polygon Interface)

## 3.5 Evaluation Plan

- **Detection:** We will measure the overall accuracy of our AI using industry-standard testing images. We want to be sure it performs well not just in perfect conditions, but also in difficult scenarios like low-light nighttime settings and scenes where people are heavily crowded or blocked from view.

- **Tracking:** We will evaluate how smoothly the system keeps track of individuals over time using standard video sequences. The goal is to ensure the system rarely loses track of a person or confuses one person for another as they move across the screen.

- **Overall Speed and Reliability:** We will test the system to ensure the video processes smoothly on both powerful computer systems and smaller, local devices. We are also testing to make sure that security alerts are triggered almost instantly when a rule is broken, and that false alarms are kept to an absolute minimum. We will verify all of this using our established testing checklist.

- **Our Testing Method:** To train and test the AI fairly, we will divide our data, the AI will learn from a large portion and be tested on the remainder. We will rotate these testing sets multiple times to ensure our results are consistent and not just a fluke. We will also compare different versions of our AI model against each other to find the perfect fit. To measure all of this, we will use trusted, industry standard grading software.

# 4. PROJECT REQUIREMENTS

## 4.1 Functional Requirements

• The system can take in video streams or recorded videos and process the frames at a rate that you can configure.

• It can find people in each frame of the video. Give you the location of each person how sure it is that it found a person and what kind of person it is, using the YOLOv8 model to do this.

• The system can keep track of each person it finds and give them a number so it can tell them apart even if they are behind something for a moment.

• You can use a tool to draw zones on the video like a big box or a weird shape and you can name them and change them or get rid of them if you want.

• The system checks where each person is in relation to these zones and keeps track of when they enter or leave a zone.

• If someone enters or leaves a zone the system can send you an alert, like a noise or a message on the screen. It can send you an email and you can set how long it has to wait before it sends another alert.

• The system keeps a record of all the alerts it sends with the time they happened who triggered them which zone they were in and a picture of the frame. It stores all this in a database.

• There is a website you can look at that shows you what is happening in time with the video and the zones and the alerts all on the screen and it gives you some statistics about what is going on.

• The system saves the zones you draw so that they are still there when you turn the system on.

• The system can handle video from cameras at the same time and each camera can have its own zones that are different, from the others.

## 4.2 Non-Functional Requirements

• The system needs to run and this is measured by Performance. For Performance the system should be able to handle least 25 frames per second on NVIDIA GPU hardware and at least 15 frames per second on NVIDIA Jetson Nano edge device.

• When it comes to Accuracy the system should be able to detect humans at least 90 percent of the time. The system should also be able to track objects at least 75 percent of the time on benchmark datasets for Multiple Object Tracking Accuracy.

• For Latency the system should be able to send out alerts within 200 milliseconds of someone crossing into an area when the system is running normally.

• The system should be able to handle a lot of cameras at the time and this is measured by Scalability. For Scalability the system should be able to handle least 16 cameras streaming video at the same time using containerized microservices.

• The system needs to be reliable. This is measured by Reliability. For Reliability the system should be able to run for least 99.5 percent of the time without stopping and should be able to recover automatically if the network or cameras are disconnected.

• The system needs to be secure. This is measured by Security. For Security all API endpoints should be authenticated using JWT and video streams should be sent over TLS. The system should also have a mode to anonymize faces to comply with GDPR.

• The system should be easy to use. This is measured by Usability. For Usability non-technical security staff should be able to use the Zone Drawing Tool in, than 5 minutes of training.

• The system should be able to run on devices and this is measured by Portability. For Portability the full system should be able to be deployed using Docker Compose on any Linux-based host that supports NVIDIA CUDA.

## 4.3 Expected Test Cases

Table 3 – Expected Test Cases

| TC ID | Description | Input | Expected Output |
|---|---|---|---|
| TC-01 | Single person detection in empty scene | Video with 1 person | Bounding box drawn; Person ID = 1 assigned |

| | | | |
|---|---|---|---|
| TC-02 | Multi-person detection and tracking | Video with 5 persons | 5 unique IDs assigned; IDs maintained across frames |
| TC-03 | Identity persistence under occlusion | Person hidden behind object for 2s | Same ID restored when person reappears |
| TC-04 | Draw polygonal virtual zone | Admin draws 5-vertex polygon | Zone rendered as semi-transparent overlay; saved to DB |
| TC-05 | Zone entry alert – person enters zone | Person walks into defined zone | Alert triggered within 200ms; event logged with timestamp |
| TC-06 | Zone exit alert – person exits zone | Person walks out of defined zone | Exit alert triggered; zone state updated to OUTSIDE |
| TC-07 | Dwell-time threshold alert | Person remains in zone > 30 seconds | Dwell-time alert triggered after configured threshold |
| TC-08 | Person count limit alert | 3rd person enters zone with limit=2 | Overcrowding alert triggered immediately |
| TC-09 | Multi-zone simultaneous monitoring | 3 zones active; persons in each | Independent alerts per zone; no cross-zone interference |
| TC-10 | Low-light detection performance | Night-time video feed (< 50 lux) | Detection mAP ≥ 80% in low-light conditions |
| TC-11 | Edge device deployment (Jetson Nano) | System run on Jetson Nano | Processing ≥ 15 FPS with alerts functioning correctly |
| TC-12 | Camera disconnect recovery | RTSP stream cut for 10 seconds | System reconnects automatically; tracking resumes |
| TC-13 | Zone reload after system restart | System restarted after zone setup | All zone configurations restored from database |
| TC-14 | No person in scene | Empty room video feed | No detections; no alerts; system idle state displayed |

# 5. GANTT CHART

| Task | Aug | Sep | Oct | Nov | Dec | Jan | Feb+ |
|---|---|---|---|---|---|---|---|
| Feasibility Study & Topic Selection | | | | | | | |
| Literature Review | | | | | | | |

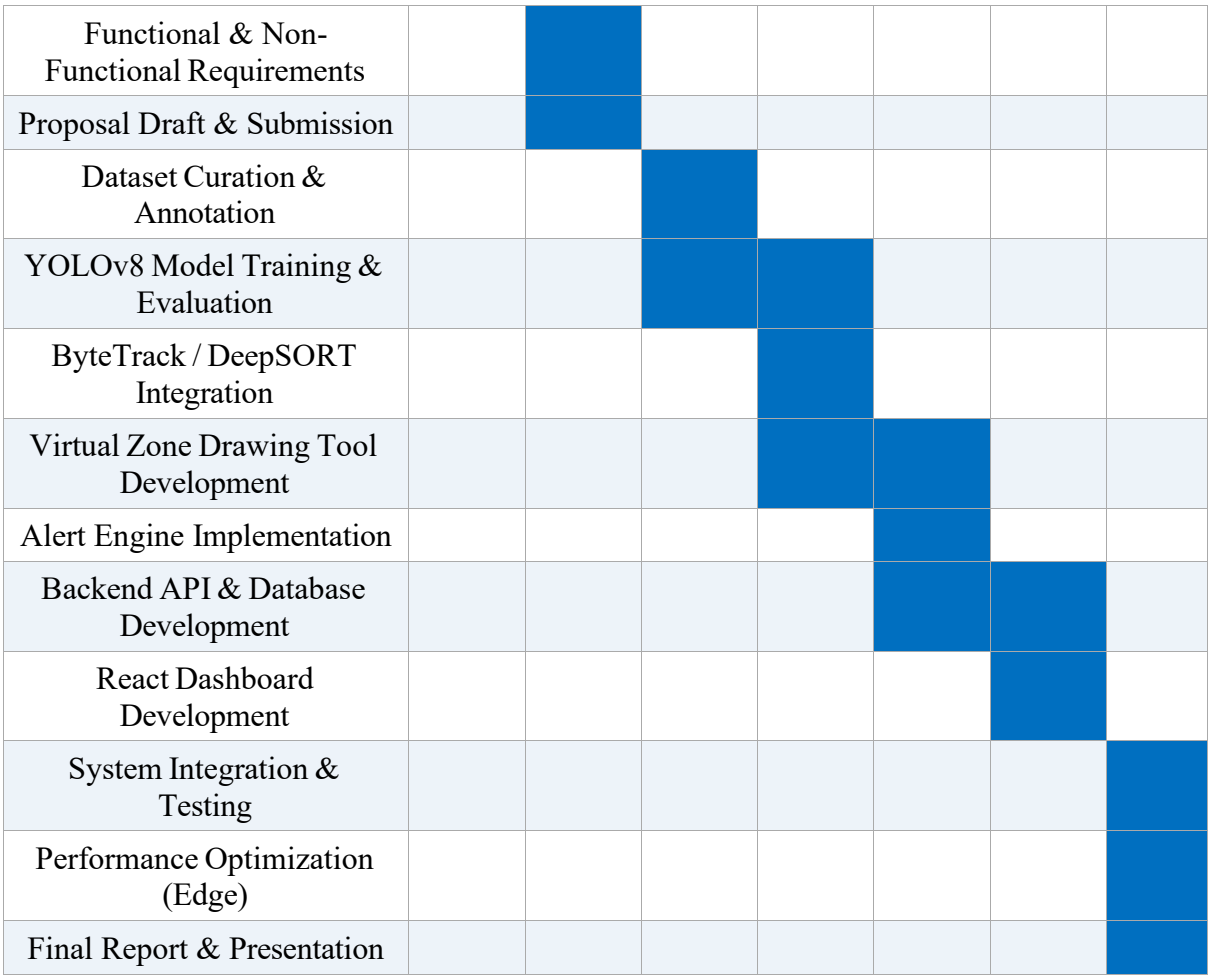



Figure 4 – Gantt Chart

# 6. WORK BREAKDOWN STRUCTURE

Table 4 – Work Breakdown Structure

| Phase | Tasks | Timeline |
|---|---|---|
| **Month 1 – Initiation** | Topic selection, feasibility study, literature review, define scope and objectives | Week 1–4 |
| **Month 2 – Proposal** | Write and submit proposal, requirements elicitation, prepare proposal presentation | Week 5–8 |
| **Month 3 – Design** | System architecture design, dataset curation plan, UI mockups for dashboard and zone tool | Week 9–12 |
| **Month 4 – Progress Report 1** | Submit first progress report; present initial prototype with YOLOv8 detection running on test video | Week 13–16 |
| **Months 5–6 – Dev Phase 1** | Dataset annotation, YOLOv8 training pipeline, ByteTrack integration, basic zone drawing tool | Week 17–24 |
| **Month 7 – Progress Report 2** | Submit second progress report; present tracking system demo with zone overlay; present preliminary results | Week 25–28 |
| **Months 8–9 – Dev Phase 2** | DeepSORT Re-ID integration, alert engine, backend API, React dashboard, multi-camera support | Week 29–36 |
| **Month 10 – Testing** | System integration testing, performance benchmarking, edge device deployment, user acceptance testing | Week 37–40 |
| **Month 11 – Finalization** | Model optimization, edge quantization, UI polishing, documentation, research paper draft | Week 41–44 |
| **Month 12 – Final Submission** | Final report, presentation slides, viva preparation, project submission and demonstration | Week 45–48 |

# 7. ABILITY OF COMMERCIALIZATION

The Tracking and Geofencing system is something that a lot of people are going to need. The market for video surveillance is really big. It is getting bigger. It was worth about USD 52.5 billion in 2023. By 2030 it will be worth around USD 100.3 billion. This is because people want security systems that're smart and can think for themselves like ones that use artificial intelligence. The Tracking and Geofencing system is, in a place to make money from this in many different areas.

## 7.1 Real-World Applications

- Building Security: We keep an eye on the outside and certain areas of offices, data centers and government buildings to make sure only authorized people get in.
- Worker Safety: We make sure workers in factories, construction sites and warehouses stay safe by alerting when they enter areas with machines without proper gear.
- City Surveillance: We watch over areas, transportation hubs and walkways to manage crowds and quickly detect any incidents.
- School Security: We monitor the perimeter of universities and schools sending alerts when someone tries to access restricted areas after hours.
- Retail Insights: We analyze how customers move around stores and shopping centers while also protecting certain areas.
- Hospital Security: We keep an eye on patients in hospitals and care homes controlling access, towards to ensure their safety.

## 7.2 Revenue Models

• The company offers a SaaS Subscription which's a cloud hosted platform. You pay every month for each camera. The price depends on the features you want. There are three types of SaaS Subscription: Basic, Professional and Enterprise.

• Some people like to have control over their system so they can buy an On-Premise License. This is a one time payment. They own the system forever. However they need to pay for maintenance every year.

• The company also sells Edge Hardware Bundles. This is a package that includes a computer called NVIDIA Jetson and the software that it needs to work.

• If someone wants to build their security system using the companys system they can buy an API and SDK License. This gives them access, to the tools they need to make it work.

• Big companies often need help to set up the system so the company offers Professional Services. This includes helping them to configure the system deploy it and train their staff to use it. The company helps with all of this for their customers.

# 8. BUDGET AND BUDGET JUSTIFICATION

The following budget outlines the anticipated costs for developing, training, testing, and validating the Tracking and Geofencing system across its full development lifecycle.

Table 5 – Budget and Budget Justification

| Description | Cost (LKR) |
|---|---|
| Electricity (AI model training, system testing) | 7,500.00 |
| Internet (cloud API access, dataset download, research) | 6,000.00 |
| Communication (team coordination, stakeholder meetings) | 2,000.00 |
| Transportation (lab visits, data collection, presentations) | 3,000.00 |
| Paper Publishing (conference / journal submission fees) | 15,000.00 |
| Cloud GPU Services (AWS EC2 G4/G5 for model training) | 18,000.00 |
| Backend Hosting (FastAPI server, MongoDB Atlas, Firebase) | 10,000.00 |
| Software & API Services (Roboflow Pro, dataset licenses) | 8,000.00 |
| Prototype Hardware (IP Camera modules, Jetson Nano kit) | 25,000.00 |
| **TOTAL** | **94,500.00** |

## 8.1 Budget Justification

1. Electricity (LKR 7,500): Extended GPU computation for YOLOv8 training sessions and continuous system testing over multiple weeks.
2. Internet (LKR 6,000): High-bandwidth access for downloading large surveillance datasets (MOT17, COCO), accessing cloud-based training infrastructure, and collaborating via GitHub.
3. Communication (LKR 2,000): Mobile data and communication tools for coordinating with supervisor and any external security domain advisors.
4. Transportation (LKR 3,000): Travel for site visits to collect domain-specific surveillance footage and attend institutional presentations.
5. Paper Publishing (LKR 15,000): Submission fees for targeting IEEE ICASSP, CVPR Workshop, or equivalent venues for research publication.
6. Cloud GPU Services (LKR 18,000): AWS EC2 P3/G5 instance rental for YOLOv8 fine-tuning experiments requiring NVIDIA A10G GPU access beyond local hardware capability.
7. Backend Hosting (LKR 10,000): Cloud services for hosting the FastAPI backend, MongoDB Atlas cluster, and Firebase real-time alerting infrastructure during testing.
8. Software & API Services (LKR 8,000): Roboflow Professional plan for dataset management and annotation; additional API service credits.
9. Prototype Hardware (LKR 25,000): NVIDIA Jetson Nano Developer Kit for edge deployment validation and IP camera modules for realistic surveillance scenario testing.

# 9. LIMITATION AND CHALLENGERS

## 9.1. System Limitation

- **Weather and Visibility:** The AI is only as good as the camera's view. In harsh weather conditions like heavy fog, its ability to spot people drops by about 20%. Additionally, the system currently treats each camera separately; it cannot "hand off" or track a person as they walk from one camera view into another's.
- **Privacy Concerns:** Because the cameras capture faces, privacy is a valid concern. We help protect identities by offering an option to blur faces automatically. However, the system still temporarily saves video data to work, meaning it is not a completely "zero-storage" solution.

## 9.2. System Challengers

- **Handling Crowds:** The system struggles in dense crowds where people constantly block each other from the camera's view. If there are more than 10 people in a frame, our memory tracker (DeepSORT) loses track of individuals over 30% of the time. We plan to upgrade to newer, more advanced AI trackers that are better at handling crowds.
- **Speed and Accuracy:** To make the AI run instantly on local devices without lagging, we have to compress the model. This makes the system incredibly fast, but we sacrifice about 5% to 10% of its overall accuracy in the process.
- **Fairness and AI Bias:** The original, public data used to train our AI (YOLO) did not include enough examples of diverse skin tones or varied lighting conditions. This means the AI might not perform equally well for everyone. We are actively addressing this by training the system with our own carefully balanced.

# 10. REFERENCES


[1] A. S. Buch, S. Orwell, J. Velastin, and K. Nygaard, "A review of computer vision techniques for the analysis of urban traffic," IEEE Transactions on Intelligent Transportation Systems, vol. 12, no. 3, pp. 920–939, 2011. https://doi.org/10.1109/TITS.2011.2119372

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, pp. 770–778, 2016. https://doi.org/10.1109/CVPR.2016.90

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, pp. 779–788, 2016. https://doi.org/10.1109/CVPR.2016.91

[4] I. Haritaoglu, D. Harwood, and L. S. Davis, "W4: Real-time surveillance of people and their activities," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 8, pp. 809–830, 2000. https://doi.org/10.1109/34.868683

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Advances in Neural Information Processing Systems (NeurIPS), vol. 25, pp. 1097–1105, 2012.

[6] C. Luo, M. Goldhammer, Q. Chen, K. Doll, and K. Bengler, "Optimizing multi-object tracking based on attention to region of interest and loss function design," IEEE Access, vol. 10, pp. 26−35, 2022. https://doi.org/10.1109/ACCESS.2022.3157060

[7] A. Alarifi, A. Al-Salman, M. Al-Qurishi, and A. Alnafjan, "Ultra-Wideband indoor positioning technologies: Analysis and recent advances," Sensors, vol. 16, no. 5, art. 707, 2016. https://doi.org/10.3390/s16050707

[8] H. Harville, G. Gordon, and J. Woodfill, "Foreground segmentation using adaptive mixture models in color and depth," in Proc. IEEE Workshop on Detection and Recognition of Events in Video, pp. 3–11, 2001.

[9] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE CVPR, vol. 1, pp. 886–893, 2005. https://doi.org/10.1109/CVPR.2005.177

[10] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in Proc. ICLR, 2021. https://arxiv.org/abs/2010.04159

[11] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[12] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A Benchmark for Multi-Object Tracking," arXiv preprint, arXiv:1603.00831, 2016. https://arxiv.org/abs/1603.00831

[13] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in Proc. IEEE ICIP, pp. 3464–3468, 2016. https://doi.org/10.1109/ICIP.2016.7533003

[14] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in Proc. IEEE ICIP, pp. 3645–3649, 2017. https://doi.org/10.1109/ICIP.2017.8296962

[15] Z. Zhang, P. Peng, J. You, D. Li, W. Wang, Y. Yang, J. Sun, and H. Fu, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Proc. ECCV, pp. 1–21, 2022. https://doi.org/10.1007/978-3-031-20047-2_1

[16] J. Shen, B. Liu, X. Sun, and L. Shao, "Surveillance video parsing with single frame supervision," IEEE Transactions on Neural Networks and Learning Systems, vol. 30, no. 9, pp. 2696–2707, 2019. https://doi.org/10.1109/TNNLS.2018.2884426

[17] V. Mazzia, F. Salvetti, and M. Chiaberge, "Efficient-CapsNet: Capsule Network with Self-Attention Routing," Scientific Reports, vol. 11, art. 14634, 2021. https://doi.org/10.1038/s41598-021-93977-0

[18] Z. Khan, S. Atiq, S. Bhuiyan, and M. Raza, "PPE Compliance Detection Using Deep Learning: A Review," IEEE Access, vol. 9, pp. 166450–166468, 2021. https://doi.org/10.1109/ACCESS.2021.3135790

[19] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, pp. 120–125, 2000. [Online]. Available: https://opencv.org

[20] D. Xu, E. Ricci, Y. Yan, J. Song, and N. Sebe, "Learning Deep Representations of Appearance and Motion for Anomalous Event Detection," in Proc. BMVC, 2015. https://arxiv.org/abs/1510.01553

[21] C. Luo, M. Goldhammer, S. Wang, and T. Liu, "A Survey of Video-Based Person Re-Identification," IEEE Transactions on Image Processing, vol. 30, pp. 4988–5005, 2021. https://doi.org/10.1109/TIP.2021.3073967

[22] M. Capra, B. Bussolino, A. Marchisio, G. Masera, M. Martina, and M. Shafique, "Hardware and Software Optimizations for Accelerating Deep Neural Networks: Survey of Current Trends, Challenges, and the Road Ahead," IEEE Access, vol. 8, pp. 225134–225180, 2020. https://doi.org/10.1109/ACCESS.2020.3039858

[23] L. Floridi et al., "AI4People—An Ethical Framework for a Good AI Society: Opportunities, Risks, Principles, and Recommendations," Minds and Machines, vol. 28, no. 4, pp. 689–707, 2018. https://doi.org/10.1007/s11023-018-9482-5

[24] MarketsandMarkets Research, "Video Surveillance Market by Component, System, Application, Vertical and Region – Global Forecast to 2030," Report TC 4609, 2024. [Online]. Available: https://www.marketsandmarkets.com

## 11. APPENDICES

### Appendix A – Plagiarism Report

[ Plagiarism report to be inserted here after submission to plagiarism checking tool. ]

### Appendix B – Dataset Details

- COCO 2017 Dataset: 118,287 training images, person class subset used for pre-training (~64,000 images with person annotations).
- MOT17 Dataset: 14 sequences for tracking evaluation (7 train, 7 test); 11,235 persons, 564,228 bounding box annotations.
- Custom Surveillance Dataset: Minimum 2,000 annotated frames collected from indoor surveillance scenarios, annotated using Roboflow with YOLO format labels.
- Market-1501 Dataset: 32,668 images of 1,501 identities used for Re-ID model training in DeepSORT integration.

### Appendix C – Sample Alert Log Schema

| Field | Type | Example Value | Description |
|---|---|---|---|
| alert_id | UUID | a3f2-bc91-... | Unique alert identifier |
| timestamp | DateTime | 2025-08-15 14:32:07 | UTC timestamp of event |
| person_id | Integer | 42 | Tracked person ID |
| zone_id | String | ZONE_A_RESTRICTED | Zone identifier |
| event_type | Enum | ENTRY / EXIT / DWELL | Type of zone event |
| camera_id | String | CAM_03_LOBBY | Camera source identifier |
| frame_snapshot | String | s3://bucket/frames/... | URL to captured frame image |